Task 6: Design Challenge – Campus Event Registration Desktop Application

Stage 1 – Requirements

Users are students, committee staff and administrators. Screens: registration form and registration table/filter/export screen. Data: student ID, name, email, event and timestamp. Rules: required fields, valid email, valid event and one registration per student/event. Invalid and duplicate records must be rejected; loading must not freeze the UI.

Stage 2 – Screen and event flow

Use JFrame with JMenuBar (Registration, Records, Help), JPanel form with JTextField/JComboBox/JButton, JTable in JScrollPane, event filter and export action. Flow: open → choose registration or records → validate → SwingWorker → service/DAO → MySQL → show dialog/update table.

Stage 3 – Critical decisions

Validate fields in the service layer; enforce duplicate prevention with a MySQL UNIQUE(student_id,event_name) constraint; use PreparedStatement; keep database calls in SwingWorker; close JDBC resources with try-with-resources; keep credentials in environment variables.

Stage 4 – Classes and handlers

EventRegistrationApp is the view; RegistrationService validates and coordinates; RegistrationDAO performs JDBC; Registration is the model. Register handler reads and validates fields, disables button, runs insert in SwingWorker and reports result in done(). Filter handler queries in SwingWorker and updates the table model on the EDT.

Stage 5 – Implementation and tests

The source is a JDBC starter requiring MySQL Connector/J, schema.sql and DB_URL/DB_USER/DB_PASSWORD. Test valid registration, invalid email/empty fields and duplicate student/event. Verify duplicate is rejected by the database constraint.

Alternative and complexity

Loading all N=100,000 rows costs O(N) transfer and O(N) memory and can freeze the UI if performed on the EDT. Paging P rows uses O(P) row memory and O(P) transfer per page; browsing all pages still transfers O(N). LIMIT/OFFSET can slow at large offsets; indexed keyset pagination is often better. SwingWorker keeps work off the EDT.

Java Source Code

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
import java.sql.*;
import java.util.*;
import java.util.List;
```

```
public class EventRegistrationApp extends JFrame {
    JTextField sid=new JTextField(8), name=new JTextField(10), email=new JTextField(12);
    JComboBox<String> event=new JComboBox<>(new String[]{"Cultural Fest","Tech Symposium","Sports Day"});
    JComboBox<String> filter=new JComboBox<>(new String[]{"All","Cultural Fest","Tech Symposium","Sports Day"});
    DefaultTableModel model=new DefaultTableModel(new String[]{"ID","Student ID","Email","Event","Date"},0);
    JTable table=new JTable(model);
    String url=System.getenv().getOrDefault("DB_URL","jdbc:mysql://localhost:3306/campus_events");
    Connection connect() throws SQLException {
        return DriverManager.getConnection(url,System.getenv().getOrDefault("DB_USER","root"),
            System.getenv().getOrDefault("DB_PASSWORD",""));
    }
    public EventRegistrationApp(){
        super("Campus Event Registration");
        JMenuBar bar=new JMenuBar();JMenu file=new JMenu("Registration");
        JMenuItem exit=new JMenuItem("Exit");exit.addActionListener(e->dispose());file.add(exit);bar.add(file);
        JMenu records=new JMenu("Records");JMenuItem refresh=new JMenuItem("Refresh");
        refresh.addActionListener(e->loadRows(null));records.add(refresh);bar.add(records);setJMenuBar(bar);
        JPanel form=new JPanel(new FlowLayout(FlowLayout.LEFT));
        form.add(new JLabel("Student ID"));form.add(sid);form.add(new JLabel("Name"));form.add(name);
        form.add(new JLabel("Email"));form.add(email);form.add(new JLabel("Event"));form.add(event);
        JButton register=new JButton("Register");form.add(register);
        JPanel top=new JPanel(new BorderLayout());top.add(form,BorderLayout.NORTH);
        JPanel fp=new JPanel();fp.add(filter);JButton apply=new JButton("Filter");fp.add(apply);top.add(fp,BorderLayout.SOUTH);
        add(top,BorderLayout.NORTH);add(new JScrollPane(table),BorderLayout.CENTER);
        register.addActionListener(e->registerAsync(register));apply.addActionListener(e-
>loadRows((String)filter.getSelectedItem()));
        setDefaultCloseOperation(EXIT_ON_CLOSE);setSize(900,420);setLocationRelativeTo(null);loadRows(null);
    }
    void registerAsync(JButton b){
        String id=sid.getText().trim(),n=name.getText().trim(),em=email.getText().trim(),ev=(String)event.getSelectedItem();
        if(id.isEmpty()||n.isEmpty()||!em.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$")){
            JOptionPane.showMessageDialog(this,"Enter ID, name and valid email");return;}
        b.setEnabled(false);
        new SwingWorker<Void,Void>(){
            protected Void doInBackground() throws Exception{
                String sql="INSERT INTO registrations(student_id,student_name,email,event_name) VALUES(?,?,?,?)";
                try(Connection c=connect();PreparedStatement p=c.prepareStatement(sql)){
                    p.setString(1,id);p.setString(2,n);p.setString(3,em);p.setString(4,ev);p.executeUpdate();}
                return null;}
            protected void
done(){b.setEnabled(true);try{get();JOptionPane.showMessageDialog(EventRegistrationApp.this,"Registration
successful");loadRows(null);}
                catch(Exception x){JOptionPane.showMessageDialog(EventRegistrationApp.this,"Registration failed / duplicate:
"+x.getMessage());}}
        }.execute();
    }
    void loadRows(String ev){
        new SwingWorker<List<Object[]>,Void>(){
            protected List<Object[]> doInBackground() throws Exception{
                List<Object[]> rows=new ArrayList<>();
                String sql="SELECT id,student_id,email,event_name,registered_at FROM registrations";
                if(ev!=null&&!ev.equals("All"))sql+=" WHERE event_name=?";
                sql+=" ORDER BY id DESC LIMIT 100";
                try(Connection c=connect();PreparedStatement p=c.prepareStatement(sql)){
                    if(ev!=null&&!ev.equals("All"))p.setString(1,ev);
                    try(ResultSet r=p.executeQuery()){while(r.next())rows.add(new
Object[]{r.getInt(1),r.getString(2),r.getString(3),r.getString(4),r.getTimestamp(5)});}}
                return rows;}
            protected void done(){try{model.setRowCount(0);for(Object[] row:get())model.addRow(row);}
                catch(Exception x){JOptionPane.showMessageDialog(EventRegistrationApp.this,"Load failed: "+x.getMessage());}}
        }.execute();
    }
    public static void main(String[] a){SwingUtilities.invokeLater(()->new EventRegistrationApp().setVisible(true));}
}
```

Input Files

sample_input.txt

Student ID: 1924260031
Name: Murtaza Khan
Email: student@example.com
Event: Cultural Fest
Repeat same student and event to test duplicate rejection.

schema.sql

```
CREATE DATABASE IF NOT EXISTS campus_events;
USE campus_events;
CREATE TABLE IF NOT EXISTS registrations (
 id INT AUTO_INCREMENT PRIMARY KEY,
 student_id VARCHAR(30) NOT NULL,
 student_name VARCHAR(100) NOT NULL,
 email VARCHAR(150) NOT NULL,
 event_name VARCHAR(100) NOT NULL,
 registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT uq_student_event UNIQUE(student_id,event_name)
);
```

Output Screenshot

Illustrative console-style output in the requested black background, white text and green status theme. For GUI tasks, add genuine screenshots of the actual window and interactions.



Compile and Run

Open terminal in src, then run:
javac EventRegistrationApp.java
java EventRegistrationApp

Task 6 requires MySQL and MySQL Connector/J. Execute input/schema.sql first and set DB_URL, DB_USER and DB_PASSWORD environment variables. Add Connector/J JAR to the classpath.